



Implementation of a Low Power IC for Neuromorphic Computing

ESE 441 Senior Design Final Project Report

Spencer Wu (EE)
Arhaam Hossain (EE)
Huabin Wu (CE)
Ryan Lin (CE)

Faculty Advisor: Milutin Stanacevic

*College of Engineering and Applied Sciences
Electrical & Computer Engineering Department
Stony Brook University*

This report summarizes the development of a Spiking Neural Network (SNN) encoding framework optimized for a low-power, resistive random-access memory (ReRAM) based neuromorphic integrated circuit (IC) for keyword spotting using the Google Speech Commands Dataset v0.02. Due to the inherent complexities associated with training native SNNs from scratch, the final implementation leverages SpikingJelly’s ANN-to-SNN (ANN2SNN) conversion pipeline to provide a viable design path. The framework adopts rate coding with Integrate-and-Fire (IF) neurons, enabling reliable threshold calibration, straightforward hardware mapping, and improved power efficiency compared to more timing-sensitive encoding schemes.

The framework utilizes a Convolutional Neural Network (CNN) architecture first before converting it into an SNN via the previously mentioned ANN2SNN API. Additionally, weights and input data were optimized through 8-bit quantization to improve efficiency and performance. These neural layers are translated onto a ReRAM crossbar utilizing differential paired inputs. In this layout, synaptic weights, initially defined as conductances ranging from 1 μS to 40 μS , are mapped to discrete memristive states, specifically a Low Resistance State (LRS) of 10 k Ω and a High Resistance State (HRS) of 300 k Ω . This implementation takes up 3 (256×256) cores to fully implement.

The input stage of the circuit is processes spectrogram data through a specialized two-phase 8-bit MAC protocol. Under this design, 8-bit pixels are split into 4-bit Most Significant Bit (MSB) and 4-bit Least Significant Bit (LSB) segments. These segments undergo integration to produce a digital Matrix-Vector Multiplication (MVM) output, where the aggregated MSB and LSB results are summed with the bias and compared against a calibrated threshold to determine whether the neuron will fire, resulting in a 1-bit value. This mechanism allows 8-bit inputs to be converted to 1-bit outputs, making the following layers function as pure SNN components, processing data as spike trains rather than multi-bit values.

II. BACKGROUND RESEARCH

A. Introduction to Neuromorphic Computing

Von Neumann architectures are defined as architectures consisting of a CPU, memory, and input/output devices. These architectures are commonly found in everyday devices such as computers, smartphones, and servers. However, despite its usage, von Neumann architectures face limitations due to the ‘memory wall’ [1]. This restriction, known as the von Neumann bot-

tleneck, is defined as the physical separation between the CPU and memory bus. This separation means that data must be constantly transferred between these components, leading to high energy consumption and delay [2]. While architectures such as the Harvard architecture address the von Neumann bottleneck, it still maintains a fundamental physical separation between instruction and data storage [3]. As a result, neuromorphic computing, a computing approach modeled after the human brain, has been discussed as a promising alternative [1, 4].

The fundamental difference between neuromorphic systems and traditional ones is the usage of artificial neurons and synapses, replicating the human brain, to process information with high efficiency. This approach allows asynchronous and event-driven computation, making it more energy efficient than systems that are synchronous and clock-driven [1]. Spiking Neural Networks (SNNs), often considered the third generation of artificial neural networks, play as a core component of neuromorphic computing by offering a more biologically plausible and power efficient model through its representation and transmission of information as ‘spike trains’, mimicking the action potentials observed in biological neurons [1, 4]. These neurons ‘fire’ only after crossing a specific activation threshold, resulting in sparse activity and significantly lower power consumption than traditional Artificial Neural Networks (ANNs) [4]. This behavior makes it excel at handling complex, dynamic, and noisy data seen in audio processing and speech recognition [4, 5].

B. Spiking Neural Network Fundamentals

Individual neurons within Spiking Neural Networks (SNNs) are typically modeled using simplified mathematical frameworks to balance biological realism with computational cost. The Integrate-and-Fire (IF) model is frequently adopted for its efficiency, however, it lacks the biological realism from the Leaky Integrate-and-Fire (LIF) model because its membrane potential does not decay over time [1, 4, 6]. In these frameworks, a neuron integrates incoming synaptic currents until its membrane potential crosses a predefined threshold, triggering a spike and an immediate potential reset [1, 6]. The choice of the IF model over the more biologically accurate LIF model is mainly preferred during hardware implementation as the ‘leak’ in an LIF neuron introduces a temporal dependency regardless of whether input spikes are received. While models like Izhikevich offer superior biological realism, the primary barrier to their adoption remains the hardware itself. The requirement for external components like multipliers makes physical implementation significantly more challenging than the IF model [1, 7]. Ultimately, the choice of model involves

a trade-off: simpler models are generally preferred for hardware implementations to minimize power consumption and silicon area [6].

Fundamentally, SNNs require effective mechanisms to encode analog input signals into spike trains processed over discrete number of timesteps, usually denoted as a hyperparameter `snn_timesteps`. The two primary methods are rate coding and temporal coding. Rate coding represents information via the firing frequency of neurons within a defined simulation window, offering enhanced training stability, precise tuning, and significantly shorter training durations. In contrast, temporal coding encodes information in the precise timing of individual spikes. The intensity component of the input is represented in a time-to-first-spike (TTFS) scheme where stronger values will spike earlier on in the spike train. While more complex, temporal coding can convey more information with fewer spikes, potentially leading to higher energy efficiency. [1, 4, 5]. Selection is often dictated by the practical feasibility of the design cycle. Figure 1 showcases the SNN pipeline that was applied for this project.

C. Learning Mechanisms in SNNs

In a typical SNN pipeline, raw audio waveforms are transformed into time-frequency representations such as Mel-Frequency Cepstral Coefficients (MFCCs). These parameters are then encoded into spike representations suitable for neuromorphic hardware. To enable keyword spotting on these neuromorphic ICs, SNNs typically require labeled datasets to map these complex auditory features to specific spoken words [8]. These applications mainly rely on supervised learning mechanisms, where the network is trained using both the input audio data and the correct labels to classify unseen data [9].

Although unsupervised learning mechanisms such as spike-timing-dependent plasticity (STDP) are more biologically plausible, they are fundamentally limited in classification tasks [10]. Unsupervised STDP extracts and clusters correlated features as its form of learning. However, this makes it difficult to differentiate similar sounding words [9]. As such, this learning mechanism is often impractical for real-world speech recognition applications that demand accurate keyword identification [8].

Direct training of pure SNNs, whether through supervised or unsupervised mechanisms, is notoriously difficult due to the properties of spiking dynamics. The nature of spike generation prevents the direct application of standard gradient-descent backpropagation [11]. While surrogate gradients have been introduced to assist in supervised training, gradient mismatch and vanishing

gradients are common issues that appear in these implementations [12]. These optimization challenges make ANN2SNN conversion widely preferred. By training an ANN using established optimization algorithms and mapping the trained weights to an equivalent SNN architecture, the conversion approach bypasses the instability of direct spike training [13]. This methodology ensures that the neural network maintain high accuracy for keyword spotting tasks while still maintaining event-driven energy efficiency properties found in other neuromorphic computing applications [14]. A native SNN approach was initially attempted for this work, as processing raw spike train inputs was preferred to maintain a purely neuromorphic pipeline. However, due to the training issues mentioned and experienced during development, the architecture was pivoted to an ANN2SNN design. This conversion simplifies the input by allowing the neural network to directly process a grid of pixels of the spectrogram representing the audio file, eliminating the need for discrete spike-train encoding. It should be noted that most conversion pipelines are mainly optimized for IF neurons as they can be linearly mapped to ReLU activation functions. Further details regarding this architectural shift are discussed in section B.

D. Hardware Architectures and Implementations

The hardware architecture implemented takes inspiration from NeuRRAM, a state-of-the-art ReRAM-based compute-in-memory (CIM) chip that can reconfigure CIM cores for multiple architectures [15]. The crossbar array operates using a one-transistor, one-resistor (1T1R) cell configuration with a neuron circuit functioning as both an analog integrator and an analog-to-digital converter (ADC). As the final SNN model is done through an ANN2SNN conversion of a CNN, which is a feedforward topology, only forward matrix-vector multiplication (MVM) of the NeuRRAM is applied for this application. The rationale for selecting a CNN-based architecture for this auditory processing task is detailed in section B.

During forward matrix-vector multiplication (MVM) within the NeuRRAM architecture, the memory lines are explicitly assigned functional roles: bit lines (BLs) serve as the voltage inputs, word lines (WLs) act as gate drivers to enable the access transistors, and source lines (SLs) act as the output channels that collect the resulting analog currents [15]. The MVM is executed physically at the device level via Ohm's law ($I = V \times G$), where the voltage on the SLs is determined by the programmed conductance (G) of the ReRAM cell. Although the final hardware deployment will rely on said programmed ReRAM conductances, the recent imple-

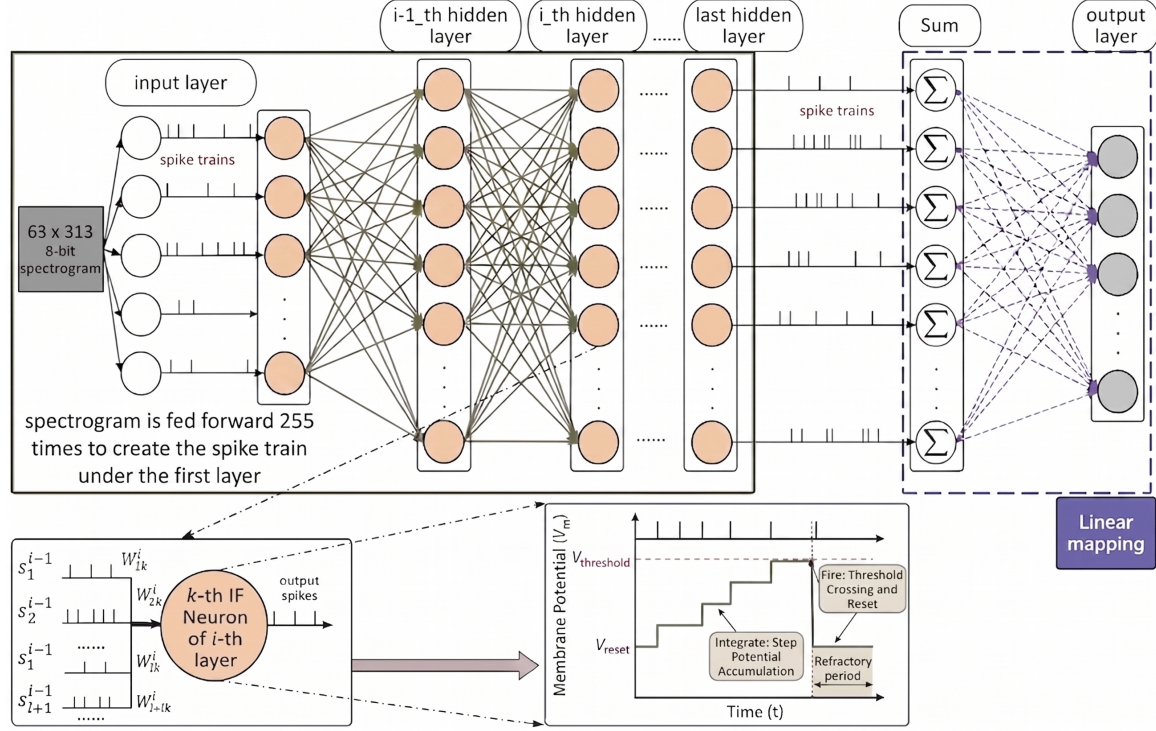


Figure 1: SNN Pipeline Architecture of Software Simulation

mentation of the circuit employs discrete fixed resistors to simulate these states. Specifically, to test the crossbar functionality, a LRS and HRS are modeled using 10 k Ω and 300 k Ω resistors, respectively. The assignment of these states within a differential pair depends entirely on the synaptic weights extracted from the neural network and converted into conductance values, with a higher conductance within the pair being mapped to a LRS and vice versa. The conversion into conductance values had a range between 1 μ S to 40 μ S, which was selected based on previous experiments with a CNN architecture [15].

In general, this design is primarily defined by two key design choices: differential pair representation and sensing mode. In software-based neural networks, weights can take both positive and negative values without any physical constraints. However, this is not the case for hardware implementations, since negative-valued weights would correspond to negative memristor conductance, which are physically improbable. To overcome this limitation, weights are encoded using differential pairs, typically denoted as $G+$ and $G-$.

There are two main approaches in implementing these differential pairs: differential output and differential input. In the differential output scheme, pairs are organized along columns, while in the differential input scheme, pairs are organized along rows. Due to their layout, differential output implementations require additional external circuitry to combine the difference be-

tween each pair. Thus, differential input configurations are generally more favorable, reducing circuit complexity, power consumption, and overall hardware area [15].

Regarding the sensing mode, architectures typically employ either current-mode or voltage-mode readouts. Current-mode sensing directly measures the summed currents on the source lines. While this maintains a constant voltage across the bit lines and simplifies input driving, it requires power-hungry peripheral circuits to process the analog currents. Consequently, voltage-mode sensing, where the accumulated charge is integrated onto a capacitor and read as a voltage swing, is generally preferred for low-power ICs. This sensing mode interfaces more efficiently with the compact, voltage-based ADCs utilized in the neuron circuits [15].

Before each MVM, all BLs, SLs, and neuron circuits are precharged to a mid-rail reference voltage, V_{ref} . This precharge equalizes the array potential, preventing unintended static leakage currents through the ReRAM cells and minimizing capacitive charging latency when input pulses are subsequently applied. Furthermore, establishing this baseline is essential for the differential input scheme, as it allows the complementary input voltages to swing above and below V_{ref} [15]. Figure 2 and 3 showcases the available differential pair representations and sensing modes.

During the MVM execution, the crossbar operates across cycles to integrate the accumulated analog charge. This cyclic operation is particularly critical for the in-

put layer of the converted SNN. The first layer of this ANN2SNN network takes multi-bit data, with subsequent layers operating strictly on 1-bit binary spikes. In this design, the ReRAM cells are programmed to express 8-bit discrete conductance states. For high-precision inputs exceeding 4 bits without saturating the analog components, the hardware divides the input data into MSB and LSB segments for sequential processing. This process is known as two-phase MVM [15]. Upon the completion of the integration phase, the resulting analog voltage is sent to the ADC for a digital representation and into the source line registers. The signal then undergoes digital post-processing, where a predefined digital bias is added to the accumulated MAC result. Finally, this total is evaluated against a fixed digital threshold; if the sum exceeds the threshold, the IF neuron fires, generating a 1-bit discrete activation (1 or 0) to be propagated as the input for the subsequent network layer.

III. STATEMENT OF WORK AND TESTBED FOR VERIFICATION

The primary deliverable of this project is a fully designed neuromorphic IC capable of detecting spoken words in noisy environments with low-power consumption. This includes a detailed circuit design and physical layout ready for fabrication. Additional deliverables will include simulation results that demonstrate speech recognition accuracy and noise robustness, a verification report showing performance metrics meeting the target design specifications, and documentation of the design methodology to support future iterations or scaling to other applications.

IV. INNOVATION

The final implementation is unique in that it integrates neural network design, hardware optimization, and full custom IC implementation into a single framework. This methodology is aligned with modern compute-in-memory (CIM) architectures such as NeuRRAM [15], which demonstrate that optimizing across device, circuit, architecture, and algorithm levels is necessary to achieve high efficiency and accuracy.

The main innovation comes from the use of the SkyWater 130 nm CMOS PDK within Cadence Virtuoso to realize a custom IC design. This includes the transistor-level implementation of ReRAM cells, neuron circuits, and peripheral logic, as well as layout, design rule check (DRC), and layout versus schematic (LVS) verification. To improve design efficiency and scalability, SKILL scripting was incorporated to automate repetitive layout tasks, enforce design rules, and streamline verification. This not only accelerated development but also ensured consistency across large array structures such as the 256×256 crossbar cores (more on this in Section C). The ability to move from high-level neural network design down to verified layout and potential GDSII generation demonstrates a complete vertical integration of tools and skills, which is a distinguishing aspect of this project.

Overall, the uniqueness of this work lies in its holistic co-design approach. Neural network techniques were adapted for hardware constraints, hardware architectures were optimized for efficient computation, and advanced design tools were leveraged to manufacture the IC. This combination of software innovation, hardware implementation, and design automation results in a system that is not only functional but also scalable and aligned with emerging neuromorphic computing research.

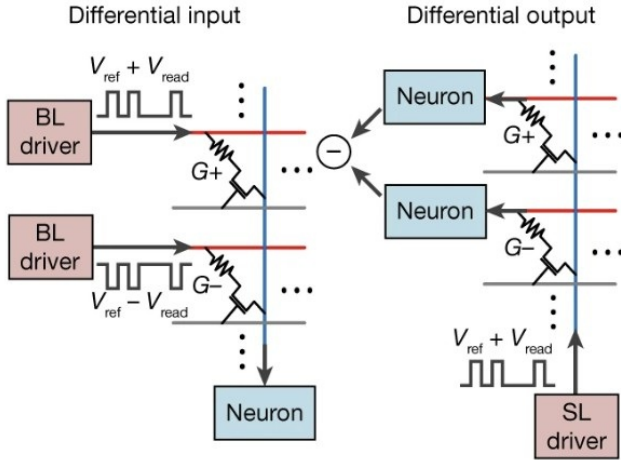


Figure 2: Differential Configurations [15]

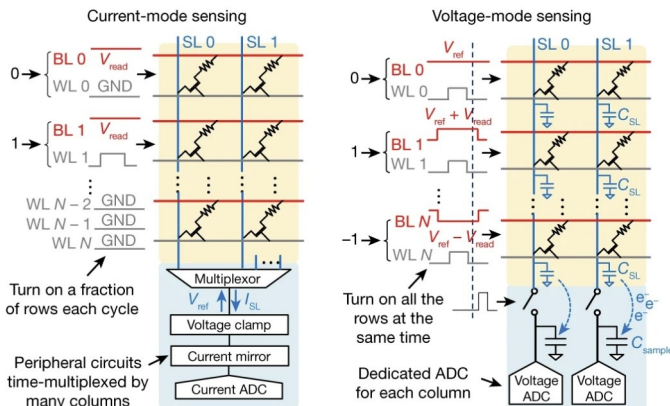


Figure 3: Sensing Modes [15]

V. CONSTRAINTS AND APPLICABLE STANDARDS

The development of the proposed neuromorphic keyword spotting system was influenced mainly by technical and financial constraints. One of the primary technical challenges was balancing neural network accuracy with hardware efficiency. Since the project targets low-power edge AI applications, the network needed to remain compact enough for hardware implementation while still maintaining strong classification performance in noisy environments. This required tradeoffs involving network depth, quantization level, neuron count, and hardware resource allocation. Additional limitations included finite memory capacity, limited bit precision, and the physical constraints imposed by the ReRAM crossbar architecture and CMOS fabrication process.

Financial constraints come from the custom IC fabrication being extremely expensive, making area efficiency and hardware simplicity important considerations throughout development. The use of the SkyWater 130 nm open-source PDK reduced development barriers while still allowing full custom IC implementation and verification. In addition, automation techniques such as SKILL scripting were used to reduce manual design effort and improve scalability.

To evaluate the tradeoffs between these constraints, several fitness criteria were established throughout the project. These included classification accuracy, power efficiency, scalability, hardware feasibility, and area utilization. Together, these metrics provided a balanced framework for evaluating the practicality and overall effectiveness of the proposed chip.

VI. BEST WORST CASE DESIGN, THEORETICAL ANALYSIS, AND SIMULATIONS

The best worst-case design strategies primarily arises from accurately modeling the transition of ReRAM cell resistive states into effective conductive values. In practical devices, the I-V characteristics are highly nonlinear, meaning the conductance varies as a function of the applied voltage. This nonlinearity introduces leakage currents and sneak-path effects that significantly degrade analog precision. To evaluate these limits, recent studies have showcased worst-case leakage scenarios by programming a single isolated cell to a High-Resistance State (HRS) while leaving all surrounding cells in a highly conductive Low-Resistance State (LRS) [16]. While prior works utilized this methodology to demonstrate sneak-path suppression in 2T-1R arrays, during the transition to conductance states, this project will adapt similar rigorous characterization strategies to evaluate and optimize the 1T1R architecture under sim-

ilarly hostile physical conditions.

Conductance relaxation, which can drift by approximately 10% of the maximum weight, introduces significant noise and variability into these analog designs. To mitigate this, previous frameworks intentionally injected Gaussian noise into the weights during software training [15]. This approach effectively trained models under worst-case hardware conditions, drastically improving resilience to device variability. Building upon this foundational strategy, this project integrates this worst-case noise-injection methodology into the ANN2SNN conversion framework, ensuring the model maintains computational stability against the specific nonlinearities and drift inherent to the hardware.

VII. PROFESSIONAL/ETHICAL ISSUES AND IMPACTS TO SOCIETY

The proposed neuromorphic IC exists within several global, economic, environmental, and societal domains. As AI and edge computing continue to grow, there is an increasing demand for low-power hardware capable of real-time inference without relying heavily on cloud infrastructure [17]. By focusing on CIM neuromorphic hardware, this project contributes toward more energy-efficient AI systems suitable for embedded and battery-powered devices. Reduced power consumption can lower operational costs while also helping address the environmental impact associated with large-scale AI workloads [18].

The project also introduces societal considerations due to its use of speech-based keyword recognition. Audio-processing systems can raise concerns regarding privacy and data security. Since the proposed architecture performs inference locally, it reduces dependence on cloud-based processing and minimizes external data transmission [19, 20]. Additionally, dataset preprocessing incorporated noisy audio conditions and balanced sampling to improve robustness.

Several ethical and professional engineering responsibilities were considered throughout development. Following principles consistent with the IEEE Code of Ethics, the project emphasized reliability, responsible technology development, and efficient system design. Engineering decisions prioritized low-power operation, scalability, and accessibility while considering the broader societal implications of AI-enabled systems [21].

The engineering decisions made throughout the project also carried broader environmental and economic impacts. The CIM architecture reduces energy consumption by eliminating data movement between memory and computation, while edge-based inference lowers communication overhead associated with remote cloud

servers [22]. As a result, many design choices influence both sustainability and long-term efficiency.

VIII. MATERIALS AND METHODS

A. Dataset

The dataset used to train the neural network originated from the Google Speech Commands Dataset v0.02 (GSCD) as it was tailored to keyword spotting applications. This collection includes 105,829 one-second audio samples of 35 common words and 6 background noise categories. As the number of recordings per word was uneven, the dataset was trimmed to ensure balance, resulting in 1,500 samples for each of the 35 spoken words. This numbered reduction was influenced by the common word with the least amount of audio recordings. Trimming was performed through four parameters to maximize diversity within the dataset: duration, average loudness, peak amplitude, and zero-crossing rate. Once the values of these four parameters were assigned to all the original audio samples, a farthest-point sampling process was used to pick the most diverse audio files.

Following the initial trimming, each of the 1,500 one-second word samples was overlaid onto a random 10-second segment of one of the six background noise. Each background noise was distributed evenly for every word, resulting in 250 samples per word were overlaid for each noise type.

The combined dataset was then split into training (80%), validation (10%), and testing (10%) sets. Crucially, this split was performed within each background noise category to maintain balanced representation. For instance, from the 250 samples containing one of the six background noise for a given word, 200 were allocated to training, 25 to validation, and 25 to testing.

The Signal-to-Noise Ratio (SNR) of each audio file was then calculated, scaling the volume of the spoken word so that the SNR falls randomly between -5dB and 20dB, ensuring robustness during training. Lastly, the peak volume of each audio file was normalized to 0.99 to prevent digital clipping and distortion.

B. Neural Network

The development of the input encoding layer followed a progressive optimization path, beginning with an exploration of temporal coding. While theoretically efficient, the approach was abandoned due to excessive training times and implementation complexity. The design then pivoted to rate coding to leverage its inherent robustness, however, standalone implementation for complex architectures remained a bottleneck for accuracy. Ultimately, a ANN2SNN conversion strategy was

selected. Specifically a CNN2SNN architecture was applied as input values were taken as spectrogram images. By utilizing the SpikingJelly API framework to automate the translation of CNN activations into spike rates, the design achieved high classification performance while ensuring the architecture remained compatible with the target hardware’s mapping protocols.

B.1. Temporal SNN Failure

A temporal encoded design was first developed. 42,000 training files were preprocessed across 33 simulation time steps using TTFS encoding. Let $\text{snn.timesteps} = T_{\text{steps}}$ the equation models for TTFS encoding of the 10-second audio files is defined as the following:

To ensure the input features map correctly to the temporal window, the code first normalizes the features.

- x_{norm} : The normalized feature value scaled to a range of $[0, 1]$, representing the relative intensity of a spectrogram pixel.
- x_{min} : The minimum intensity value found in the audio features, used to establish a baseline for subtraction.
- x_{99} : The 99th percentile feature value used for robust calibration to prevent outliers from compressing the dynamic range.

$$x_{\text{norm}} = \text{clip} \left(\frac{x - x_{\text{min}}}{x_{99} - x_{\text{min}}}, 0, 1 \right) \quad (1)$$

The core TTFS logic is an inverse linear relationship. In TTFS encoding, early behavior is strong: an intensity of 1.0 should spike at time step 0, while an intensity of 0.0 should spike at the very end of the simulation window (T_{sim}).

$$t = (1 - x_{\text{norm}}) \cdot T_{\text{steps}} \quad (2)$$

This final equation converts the continuous time value into a discrete integer step compatible with hardware simulation. It applies a floor operation and clamps the result to ensure it stays within the timestep range (0 to $T_{\text{sim}} - 1$). Altogether, the preprocessing essentially creates a "stack" of 33 different spectrograms showing how the significant temporal features of the audio evolve over the actual timeline of the audio file. This can be thought of as a topographic map looking down where the first spectrogram of the stack, or snn.timesteps , show the loudest features of the mel-frequency components. It gradually reveals the quieter features traveling down the simulation timeline. The depth at which a spike appears

becomes a direct proxy for its original decibel level. This effectively transforms a dense analog audio spectrogram into a sparse, binary structure where information is encoded by not how much voltage is applied, but precisely when the first pulse arrives.

$$t_{spike} = \text{clip}(\lfloor (1 - x_{norm}) \cdot T_{steps} \rfloor, 0, T_{steps} - 1) \quad (3)$$

The training session loop was performed across 60 epochs starting with `snn.timesteps = 33` as a scaled-down version of the original audio file. In this version, LIF neurons were utilized from `spikingjelly.activation_based`, and the mathematical model is shown as follow:

The internal state of the neuron, or membrane potential (V_m), accumulates input while simultaneously "leaking" over time. The charging equation (pre-spike potential $H[t]$) is

$$H[t] = V[t-1] + \frac{1}{\tau} (X[t] - (V[t-1] - V_{reset})) \quad (4)$$

- $V[t-1]$: The membrane potential from the previous time step.
- $X[t]$: The external input spikes at the current time step.
- τ : The time constant that controls the "leak" rate.
- V_{reset} : The potential the neuron returns to after firing.
- $V_{threshold}$: The firing threshold voltage.
- $S[t]$: The binary spike output (1 or 0).

After a spike is generated, the membrane potential is reset. SpikingJelly supports two types of resets: reset-to-zero as hard reset, and reset-by-subtraction as soft reset. Hard reset was implemented for simplification.

$$V[t] = H[t] \cdot (1 - S[t]) + V_{reset} \cdot S[t] \quad (5)$$

While a standard CNN processes an entire spectrogram image of the audio file in a single forward pass ($T_{sim} = 1$), the TTFS temporal-encoded SNN architecture requires computing this pass 33 separate times for a single audio file sample for the ideal temporal precision required as part of hardware constraints. The system is effectively performing approximately 83 million full forward and backward passes (derived from $42000 \text{ files} \times 33 \text{ steps} \times 60 \text{ epochs}$).

An average of 1.59 iterations/second per epoch was observed, equating to 14 hours to finish a full training

session. This created a bottleneck in the training simulation where the hardware is forced to behave like a network that is 33 times deeper than its physical layer count, leading to an unsustainable development cycle. The learning process is further slowed by the inherent difficulty of backpropagating through time in a SNN scheme. TTFS encoding is inherently sparse and discrete, where a neuron fires exactly once or not at all. This sparsity often leads to gradient starvation. If a neuron fails to fire during a training step, its gradient becomes zero, leaving the optimizer with minimal information on how to update the weights.

Figure 4 shows the accuracies of this TTFS SNN. While training accuracy steadily increases and loss decreases, validation accuracy remains stagnant around 22.7%, indicating severe overfitting. Furthermore, the linearly compounding simulation time reaches 11.43 hours, exposing the severe computational bottleneck of the TTFS temporal-encoding framework.

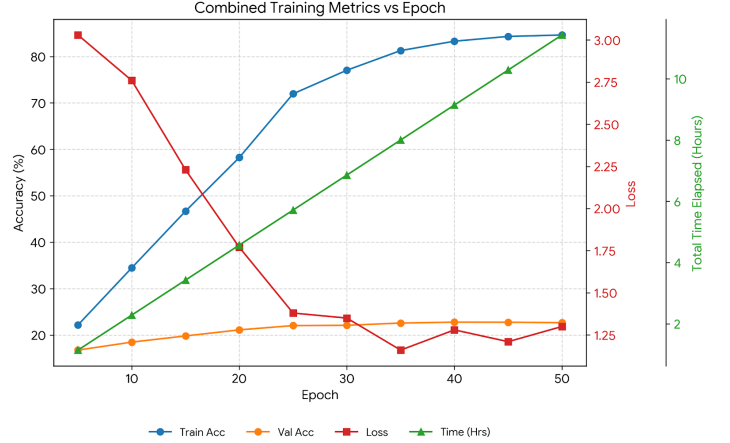


Figure 4: Combined Training Metrics for Raw TTFS SNN Training Attempt Over 50 Epochs

B.2. Rate SNN Failure

Following the failure of the temporal TTFS approach, development shifted to rate coding as a more trainable alternative. Under rate coding, each spectrogram pixel value is interpreted as a firing probability. At every simulation timestep t , a spike is generated by sampling from a Bernoulli distribution parameterized by the normalized pixel intensity:

$$S[t] \sim \text{Bernoulli}(x_{norm}) \quad (6)$$

where $x_{norm} \in [0, 1]$ is the log-normalized mel spectrogram value. Over T timesteps, the mean firing rate $\bar{r}$ of a given input neuron converges to the pixel intensity:

$$\bar{r} = \frac{1}{T} \sum_{t=1}^T S[t] \approx x_{norm} \quad (7)$$

The architecture used a convolutional SNN built on SpikingJelly’s activation-based API with $T = 201$ timesteps. The model consisted of three convolutional blocks using 64 channels each, with increasing dilation rates (1×1 , 4×3 , 16×9) and distinct LIF time constants ($\tau = 2.0, 4.0, 8.0$) intended to capture dynamics at different temporal scales. A fully connected layer mapped the convolutional output down to 35 classes. LIF neurons in each layer followed the same membrane dynamics described in Section B.1 (Equations 4 and 5), with the time constant τ varying per layer.

Training used Adam with a learning rate of $1e-2$, cosine annealing down to $1e-5$, batch size 64, across 70 epochs. Since spiking neurons are non-differentiable at the firing threshold, backpropagation requires a surrogate gradient to approximate the derivative of the Heaviside step function $\Theta(V - V_{th})$. This implementation used the sigmoid surrogate:

$$\frac{\partial S}{\partial V} \approx \sigma'(\alpha(V - V_{th})) = \frac{\alpha \cdot e^{-\alpha(V - V_{th})}}{(1 + e^{-\alpha(V - V_{th})})^2} \quad (8)$$

where $\alpha = 10.0$ controls the sharpness of the approximation. As α increases, the surrogate approaches the true Heaviside derivative (a Dirac delta), but gradients vanish for neurons far from the threshold. As α decreases, gradients become broader but less accurate.

Before accuracy could be evaluated, the model encountered NaN loss values during the first epoch. The root cause was traced to silent or corrupted audio files in the dataset that produced zero-variance mel spectrograms. When these samples entered the $\log_{10}(\text{mel}/\text{max})$ normalization step, division by zero propagated NaN values through the entire forward pass. The issue was resolved with a `torch.nan_to_num()` call before normalization.

Once training stabilized, the model initially showed promise, reaching 70% training accuracy within the first 10 epochs. However, progress slowed considerably beyond that point. After 70 full epochs, the network achieved a training accuracy of 79.5% and a validation accuracy of 76.3%, with a best validation accuracy of 76.9% at epoch 66. The final test accuracy was 77.07% with a macro F1 score of 0.77.

The primary bottleneck was surrogate gradient degradation through depth. While the sigmoid surrogate (Equation 8) provides a reasonable approximation for a single layer, the error compounds during backpropagation through L spiking layers and T temporal unrolling

steps. The effective gradient at the first layer passes through:

$$\frac{\partial \mathcal{L}}{\partial W_1} = \prod_{l=1}^L \prod_{t=1}^T \frac{\partial S_l[t]}{\partial V_l[t]} \cdot \frac{\partial V_l[t]}{\partial V_l[t-1]} \cdot \dots \quad (9)$$

With $L = 4$ LIF layers and $T = 201$ timesteps, this chain becomes extremely long. Unlike standard ReLU networks where $\partial \text{ReLU}/\partial x \in \{0, 1\}$ exactly, each surrogate factor introduces approximation error that accumulates multiplicatively. By the time gradient information reached the earliest convolutional block, the learning signal had degraded substantially.

A secondary issue was information loss from the rate coding representation itself. The averaging operation in Equation 7 smooths out temporal dynamics in the spectrogram, making acoustically similar words like "three" versus "tree" or "go" versus "no" harder to distinguish since their key differences lie in transition timing rather than overall spectral energy.

Additionally, each 70-epoch training run took approximately 11.6 hours (~ 580 seconds per epoch). This severely restricted hyperparameter exploration, as each experiment with a different learning rate schedule, time constant, or dilation configuration required a separate full-length session. Checkpoint-and-resume logic was added as a solution, but the slow feedback loop made it impractical to thoroughly search the design space within the project timeline. Figure 5 shows the accuracies of the native rate coding SNN.

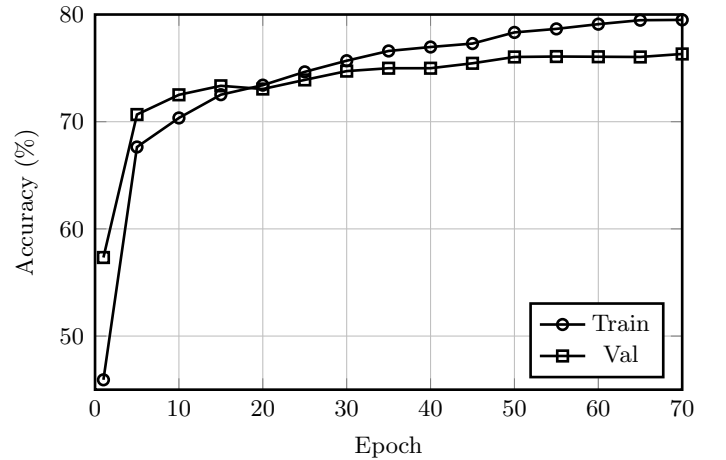


Figure 5: Native Rate Coding SNN’s Training and Validation Accuracy over Epochs

B.3. ANN2SNN Architecture and Results

After rate coding plateaued at 77%, the architecture was shifted to SpikingJelly’s ANN2SNN conversion pipeline. An initial implementation using a ResNet10

architecture was explored for the conversion due to its strong performance in standard classification tasks [23]. However, ResNet architectures rely on residual skip connections, where the output of a block is computed as:

$$y = \mathcal{F}(x, \{W_i\}) + x \quad (10)$$

This additive identity shortcut introduces non-feedforward data paths that do not map onto the ReRAM crossbar topology. The crossbar performs forward MVM only, and routing skip connection outputs back to earlier points in the computation would require additional buffering and inter-core routing that significantly complicated the hardware design. As a result, the ResNet10 was abandoned in favor of a purely sequential CNN architecture that maps directly onto a feedforward crossbar layout.

The CNN preprocessing pipeline was built using the `librosa` library. Each one-second audio sample was loaded at a sample rate of $f_s = 16$ kHz. A mel spectrogram was computed with the following parameters:

$$\mathbf{M} = \text{MelSpectrogram}(x(t), n_{\text{mels}} = 64, n_{\text{fft}} = 512, h = 160) \quad (11)$$

where $x(t)$ is the raw audio waveform, n_{mels} is the number of mel filter banks, n_{fft} is the FFT window size, and h is the hop length in samples. The spectrogram was then converted to decibel scale:

$$\mathbf{M}_{dB} = 10 \cdot \log_{10}(\mathbf{M} + \epsilon) \quad (12)$$

where ϵ is a small constant to prevent logarithm of zero. Per-sample normalization was then applied by subtracting the mean and dividing by the standard deviation:

$$\hat{\mathbf{M}} = \frac{\mathbf{M}_{dB} - \mu}{\sigma} \quad (13)$$

The resulting 2D spectrogram was treated as a single-channel image input to the CNN.

The CNN itself consisted of three convolutional blocks, each containing a 2D convolution layer, batch normalization, and a ReLU activation, followed by average pooling. The initial channel count was set to 16, doubling at each subsequent block (16, 32, 64). Kernel sizes were set to 3×3 with a stride of 2 and padding of 1 to preserve spatial dimensions before pooling. After the convolutional blocks, the feature maps were flattened and passed through a fully connected layer mapping to 35 output classes. Both the architecture depth and channel count were selected after iterative experimentation to balance classification accuracy against the total neuron count that the hardware could support. More details about this configuration can be found in Section A.

Training was performed using standard backpropagation with an Adam optimizer and cross-entropy loss. Prior to conversion, each convolution and batch normalization pair was fused into a single equivalent convolution. Given a convolution with weight W and bias b , and a subsequent batch normalization with learned parameters γ , β and running statistics μ_{BN} , σ_{BN} , the fused weight $\hat{W}$ and bias $\hat{b}$ are:

$$\hat{W} = \frac{\gamma}{\sigma_{BN}} \cdot W \quad (14)$$

$$\hat{b} = \frac{\gamma}{\sigma_{BN}} \cdot (b - \mu_{BN}) + \beta \quad (15)$$

This fusion eliminates the batch normalization layers from the graph, which is necessary as batch normalization has no direct spiking equivalent.

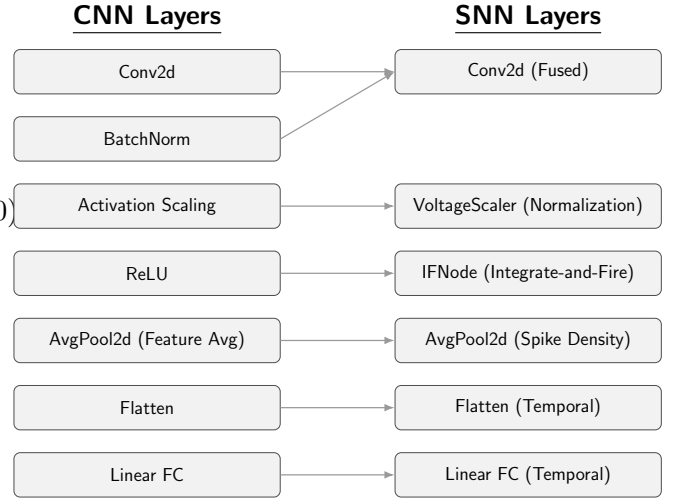


Figure 6: Layer-by-Layer Mapping and Fusion from the CNN Architecture to the SpikingJelly SNN Architecture.

SpikingJelly’s ANN2SNN converter was then applied, replacing each ReLU activation with an Integrate-and-Fire (IF) neuron as shown in Figure 6. The IF neuron follows a simplified form of the membrane equation without leak:

$$V[t] = V[t - 1] + X[t] \quad (16)$$

When $V[t] \geq V_{th}$, a spike is emitted ($S[t] = 1$) and the membrane resets. As the ReLU function $f(x) = \max(0, x)$ is a piecewise linear activation, there exists a direct mathematical correspondence: an IF neuron’s average firing rate over T timesteps converges to a scaled version of the ReLU output, provided the threshold V_{th} is properly calibrated [13]. During conversion, each IF neuron layer’s threshold was calibrated using the 99.99th percentile of activation values observed across a calibration dataset:

$$V_{th}^{(l)} = P_{99.99} \left(\{a_i^{(l)}\}_{i=1}^N \right) \quad (17)$$

where $a_i^{(l)}$ is the ReLU activation at layer l for calibration sample i , and N is the number of calibration samples. This ensures that the converted SNN’s firing rates closely match the ANN’s original activation distributions without excessive saturation.

Following conversion, the model weights and input spectrograms were quantized to 8-bit precision to align with the hardware’s conductance resolution. The continuous weight values were mapped to discrete 8-bit integers via uniform quantization:

$$w_q = \text{round} \left(\frac{w - w_{min}}{w_{max} - w_{min}} \cdot (2^8 - 1) \right) \quad (18)$$

The quantized ANN2SNN model achieved approximately 83% test accuracy on the same 35-class keyword spotting task, a 6-point improvement over the direct rate coding approach. The conversion approach eliminated the surrogate gradient instability, temporal information loss, and training time constraints that had limited the previous implementations. The converted SNN operates with IF neurons and rate coding, which maps directly onto the two-phase 8-bit MAC protocol described in Section D, where 8-bit spectrogram pixels are split into MSB and LSB segments for integration on the crossbar.

C. Hardware

One of the primary architectural decisions in the hardware implementation was whether to map the neural network onto a single large ReRAM core or partition it across multiple smaller cores. While a single large core could reduce the amount of peripheral buffering and inter-core communication, it would substantially increase both area and power consumption. Larger arrays require longer bit lines, source lines, and word lines, which increases parasitic resistance and capacitance throughout the crossbar. This leads to higher dynamic power consumption during charging and discharging of the lines, greater signal degradation, and more demanding sensing circuitry. In addition, the differential mapping scheme doubles the effective row count, making a single implementation inefficient from both a layout and decoder sizing perspective. To address these trade-offs, the network was partitioned across three separate 256×256 cores. Using smaller cores would reduce interconnect lengths, lower parasitics, and improve overall energy efficiency during MVM operations. This partitioned approach also enables more compact layouts and improved scalability while remaining within the practical area constraints of the hardware. Although the

multi-core implementation introduced additional digital recombination logic between folded layers, the reduction in routing overhead and interconnect power made the three-core architecture a more efficient solution overall. Figure 7 showcases all the layers mapped onto the three-core architecture. Note that there is a gap between rows 144 to 160 to ensure that a 32-decoder block can work.

The hardware was implemented in Cadence Virtuoso using the SkyWater 130 nm CMOS PDK. Initial neuron development began with a CMOS operational transconductance amplifier using NMOS transistors as switching elements for the control paths. However, after further analysis of the NeuRRAM reference design, the neuron implementation was modified to use a single-stage cascode CMOS amplifier as the integrator since the reference neuron utilized a single input terminal structure. To properly bias the amplifier, the bias voltage was swept from 0V to 2V through simulation in order to evaluate gain behavior, operating region stability, and output response. From these sweeps, a V_{bias} value of 0.8V was selected as the optimal operating point for stable integration behavior. During implementation, the control switches were also redesigned. Initially, NMOS switches were used for sampling and integration control, these were later replaced with CMOS transmission gates. This change was necessary as NMOS switches alone can only pass a strong logic low and are limited to passing approximately $V_{DD} - V_{TN}$ for logic high signals, which introduced voltage degradation during charge transfer. Transmission gates eliminated this issue by allowing full rail-to-rail voltage transfer during neuron operation.

Following the neuron implementation, the ReRAM crossbar arrays were constructed using NMOS access transistors and resistors to model the 1T1R cell structure. Each synaptic weight was represented using differential $G+$ and $G-$ conductance pairs to support signed weight operations within the hardware. Due to the extremely repetitive nature of the array layout, SKILL scripting was heavily utilized to automate large-scale device placement and routing generation throughout the crossbar implementation. This significantly reduced manual layout effort while ensuring consistency and alignment across the repeated array structures.

IX. RESULTS AND DISCUSSION

A. Neural Network Results

Table 1 presents a performance comparison across various initial channel configurations and layer depths. Due to the constraints of the three-core architecture, the search space was limited to the displayed parameters as configurations exceeding these values surpassed the

ROWS (BLs/WLs) X COLUMNS (SLs)

Initial Conv (18 x 16)
 Layer 1 Conv 1 (288 x 16)
 Layer 1 Conv 2 (288 x 16)
 Layer 2 Conv 1 (288 x 32)
 Layer 2 Conv 2 (576 x 32)
 Layer 3 Conv 1 (576 x 64)
 Layer 3 Conv 2 (1152 x 64)
 FC Layer (128 x 35)

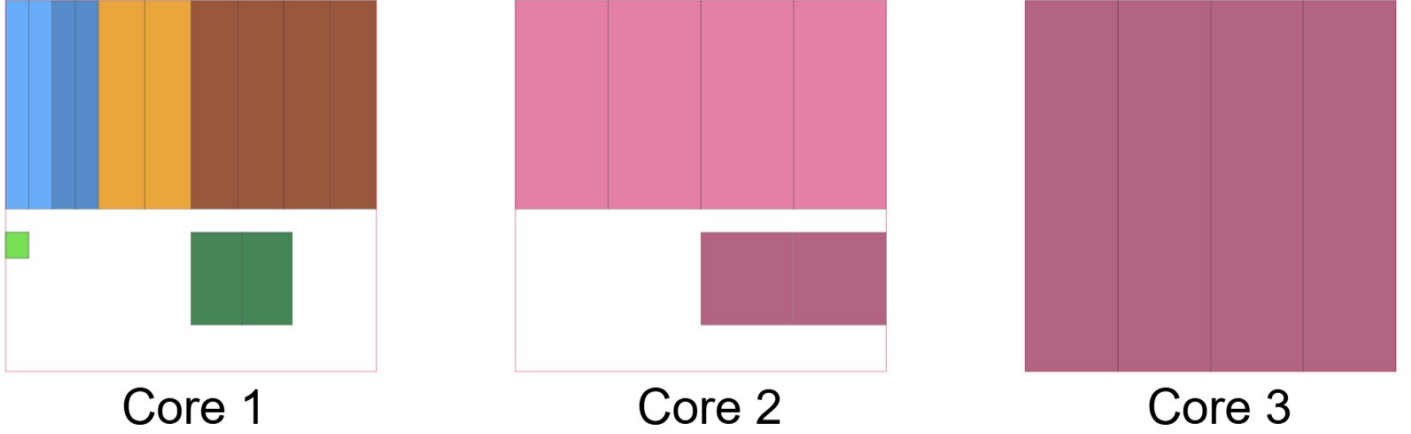


Figure 7: Three Core Block Layout

hardware’s capacity. It is important to note that the parameter counts listed represent software weights, biases, and thresholds. As a reminder, going from software to hardware implementation, the number of weights are required to be doubled to account for differential pairs. These quantized SNN accuracies were initially evaluated at 8-bits. 8-bits was selected prematurely as prior ReRAM-based neuron designs [15] indicate that voltage-mode neurons supports a maximum output resolution of 8 bits. Parameters include both the weights and biases.

Training utilized a batch size of 16 and was capped at 45 epochs, employing an early stopping mechanism with a patience of 5 epochs to prevent overfitting. Figure 8 showcases the resulting training and validation accuracies and loss over these epochs.

Table 1: Quantized SNN Performance from Configurations

Init. Ch.	Parameters	Res. Blocks	Accuracy
8	5296	2	60.08%
8	19744	3	78.53%
8	76288	4	81.18%
16	19808	2	74.17%
16	76352	3	83.12%
32	76480	2	75.18%

To determine if the appropriate quantization precision was actually 8 bits, 4-bit, 6-bit, and 8-bit settings were evaluated, with the optimal bit-width selected based on quantization accuracy. Among these, 8-bit quantization yielded the best performance. A comparison of the evaluated configurations is shown in Table 2. It should be noted that during training, the quantized spectrogram inputs were temporarily cast back to 32-bit floating-point representations for backpropagation.

Table 2: Bit Comparison for Quantization

Bit Resolution	Accuracy
4	50.50%
6	80.80%
8	83.12%

The dimensions of each layer are summarized in Table 3. Again, the number of weights displayed must be doubled to take into account differential pairs for hardware implementation.

B. Hardware Results

Figures 9 and 10 present the schematic-level implementations of the Core 1 ReRAM crossbar arrays and a

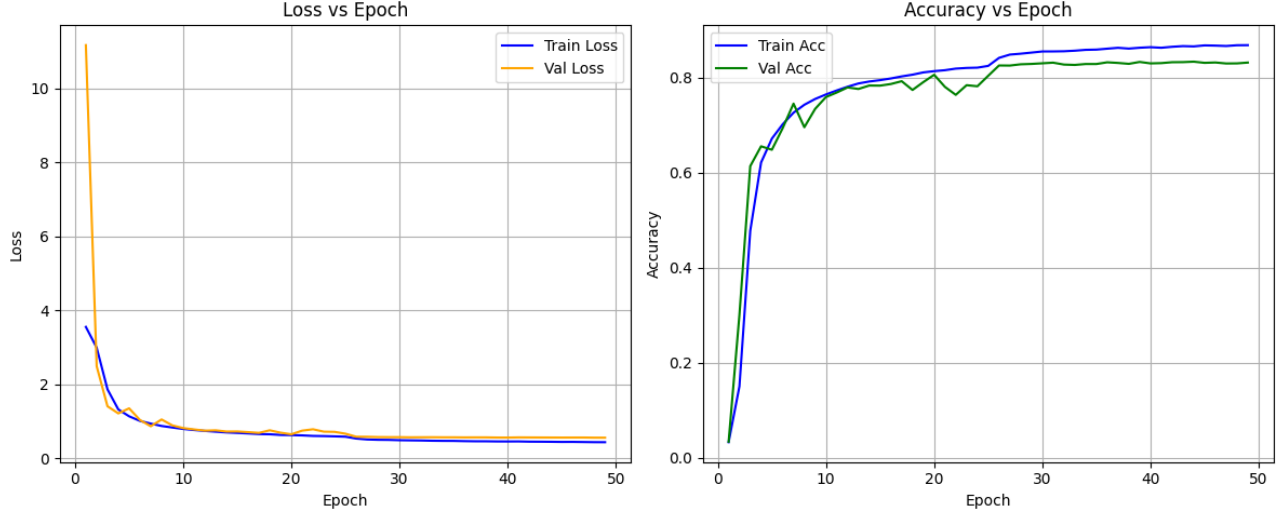


Figure 8: ANN2SNN Training Simulation Loss and Accuracy Results Over 45 Epochs and Fine-Tuning

Table 3: SNN Weight Array Sizes (16 Initial Channels, 3 Residual Blocks)

Layer	Dimensions
Initial Convolution	(9×16)
Layer 1 Convolution 1	(144×16)
Layer 1 Convolution 2	(144×16)
Layer 2 Convolution 1	(144×32)
Layer 2 Convolution 2	(288×32)
Layer 3 Convolution 1	(288×64)
Layer 3 Convolution 2	(576×64)
Fully Connected	(64×35)
Total Weights	76,112
Biases	240
Total Parameters	76,352

single ADC neuron. The ReRAM crossbar arrays are composed of 1T1R ReRAM cells utilizing differential row pairing with LRS HRS resistors to represent $G+$ and $G-$ conductance pairs. The ADC neuron implemented for each source-line consists of a capacitor, CMOS transmission gates, a single-stage cascode amplifier, latches, and multiplexers. Simulation was performed to verify correct neuron operation and confirm the expected integrator functionality. Figure 11 shows the simulated V_{in} and V_{out} waveform of the ADC neuron where V_{out} reflects the accumulated charge on the capacitor. Unlike theoretical integration, it can be seen that there are fluctuations during each switching period, mainly caused by the cyclic operation. Regardless, the overall behavior of the integration showcases ideal integration within the neuron, as the level the integration stops at is more important than the noise during switching.



Figure 9: Core 1 Schematic

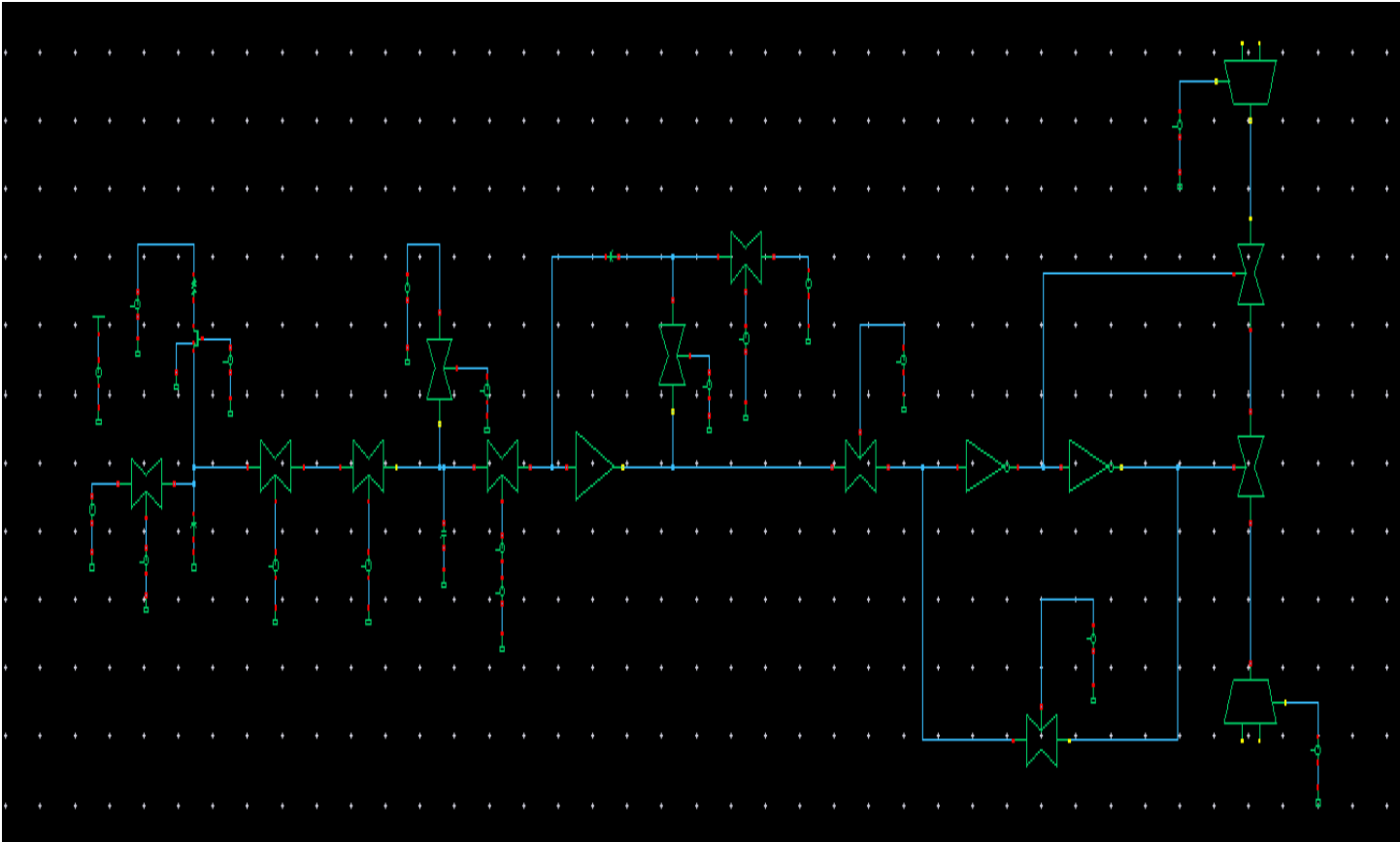


Figure 10: Neuron Schematic

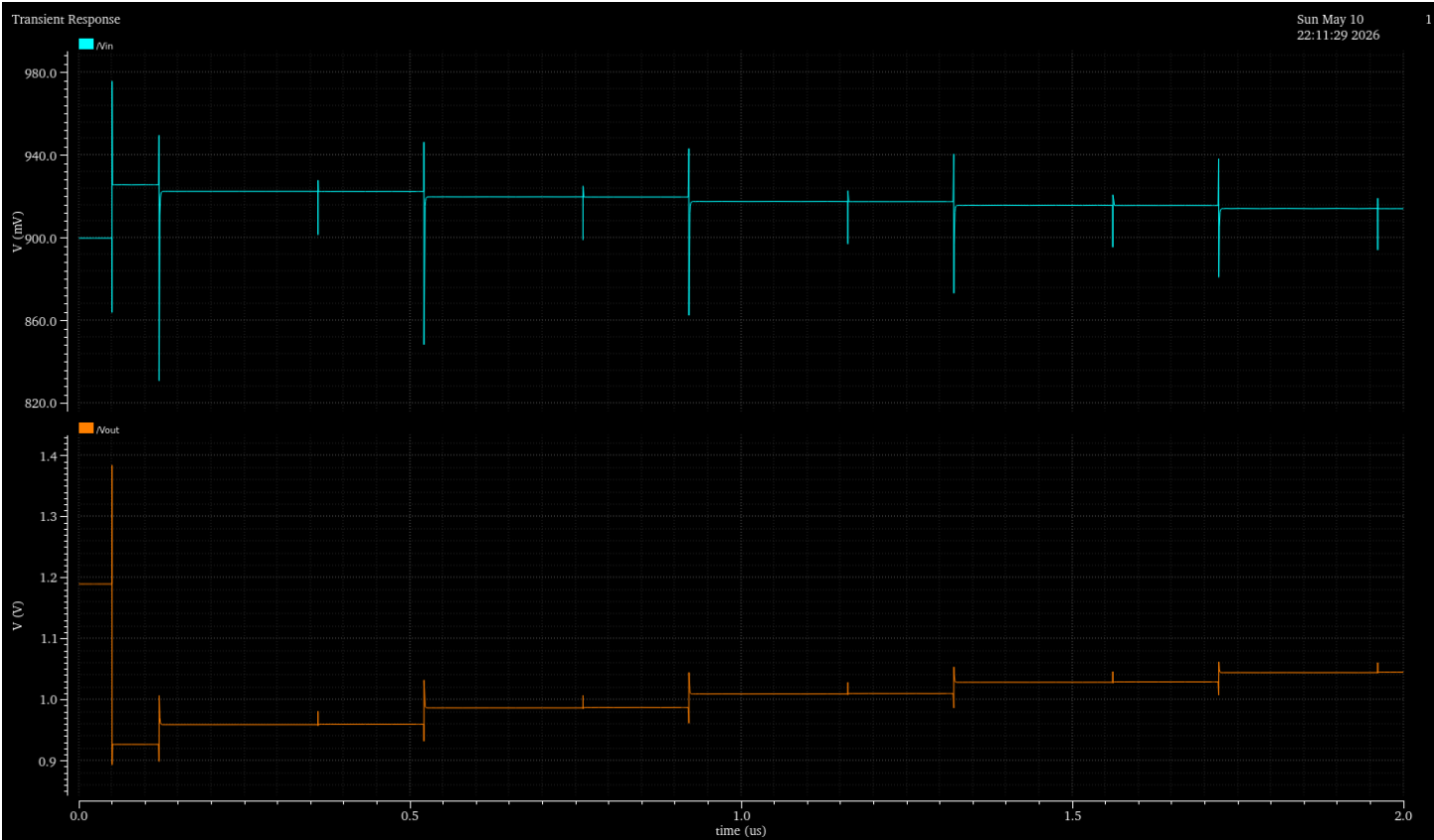
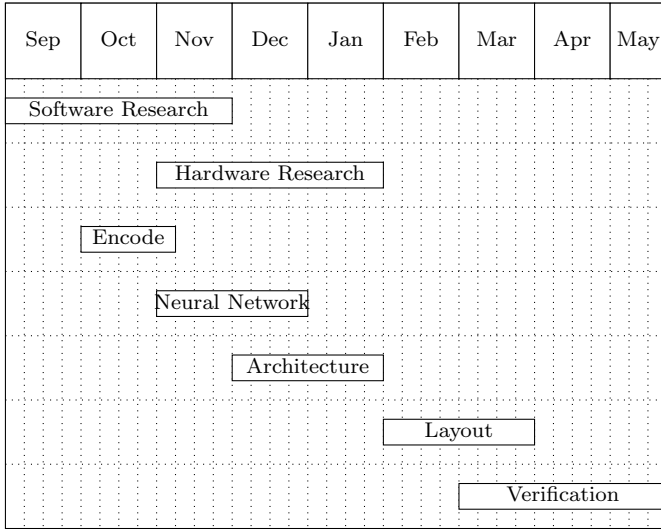


Figure 11: Neuron Waveform: V_{in} and V_{out}

REFERENCES

- [1] Amar Shrestha et al. “A survey on neuromorphic computing: Models and hardware”. In: *IEEE Circuits and Systems Magazine* 22.2 (2022), pp. 6–35.
- [2] Haitong Li et al. “A learnable parallel processing architecture towards unity of memory and computing”. In: *Scientific reports* 5.1 (2015), p. 13330.
- [3] Shuo Xue. “Overview of Computer Architecture Development Direction Breaking through Von Neumann Architecture”. In: *International Core Journal of Engineering* 7.8 (2021), pp. 330–334.
- [4] Bipin Rajendran et al. “Low-power neuromorphic hardware for signal processing applications: A review of architectural and system-level design approaches”. In: *IEEE Signal Processing Magazine* 36.6 (2019), pp. 97–110.
- [5] Nitin Rathie et al. “Exploring neuromorphic computing based on spiking neural networks: Algorithms to hardware”. In: *ACM Computing Surveys* 55.12 (2023), pp. 1–49.
- [6] Wenzhe Guo et al. “Neural coding in spiking neural networks: A comparative study for robust neuromorphic systems”. In: *Frontiers in Neuroscience* 15 (2021), p. 638474.
- [7] Thanh-Dat Nguyen et al. “Energy-Efficient Izhikevich Neuron Design Using Approximate CORDIC-Based Multipliers for Low-Power Neuromorphic Hardware”. In: *IEEE Access* 14 (2026), pp. 22146–22161.
- [8] Alexandre Bittar and Philip N Garner. “A surrogate gradient spiking baseline for speech command recognition”. In: *Frontiers in Neuroscience* 16 (2022), p. 865897.
- [9] Fangxin Liu et al. “SSTDTP: Supervised spike timing dependent plasticity for efficient spiking neural network training”. In: *Frontiers in Neuroscience* 15 (2021), p. 756876.
- [10] Gaspard Goupy, Pierre Tirilly, and Ioan Marius Bilasco. “Paired competing neurons improving STDP supervised local learning in Spiking Neural Networks”. In: *Frontiers in Neuroscience* 18 (2024), p. 1401690.
- [11] Yangfan Hu et al. “Fast-snn: Fast spiking neural network by converting quantized ann”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.12 (2023), pp. 14546–14562.
- [12] Zihan Huang et al. “Towards high-performance spiking transformers from ann to snn conversion”. In: *Proceedings of the 32nd ACM international conference on multimedia*. 2024, pp. 10688–10697.
- [13] Andrea Castagnetti, Alain Pegatoquet, and Benoît Miramond. “Trainable quantization for speedy spiking neural networks”. In: *Frontiers in Neuroscience* 17 (2023), p. 1154241.
- [14] Qu Yang, Qi Liu, and Haizhou Li. “Deep residual spiking neural network for keyword spotting in low-resource settings.” In: *Interspeech*. 2022, pp. 3023–3027.
- [15] Weier Wan et al. “A compute-in-memory chip based on resistive random-access memory”. In: *Nature* 608.7923 (2022), pp. 504–512.
- [16] Irem Didin et al. “Characterization and Modeling of Multilevel Analog ReRAM Synapses in the Sky130 Process”. In: *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits* (2026).
- [17] Raghubir Singh and Sukhpal Singh Gill. “Edge AI: a survey”. In: *Internet of Things and Cyber-Physical Systems* 3 (2023), pp. 71–92.
- [18] Emma Strubell, Ananya Ganesh, and Andrew McCallum. “Energy and policy considerations for deep learning in NLP”. In: *Proceedings of the 57th annual meeting of the association for computational linguistics*. 2019, pp. 3645–3650.
- [19] Reza Shokri and Vitaly Shmatikov. “Privacy-preserving deep learning”. In: *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*. 2015, pp. 1310–1321.
- [20] Brendan McMahan et al. “Communication-efficient learning of deep networks from decentralized data”. In: *Artificial intelligence and statistics*. Pmlr. 2017, pp. 1273–1282.
- [21] Roy Schwartz et al. “Green ai”. In: *Communications of the ACM* 63.12 (2020), pp. 54–63.
- [22] Flavio Bonomi et al. “Fog computing and its role in the internet of things”. In: *Proceedings of the first edition of the MCC workshop on Mobile cloud computing*. 2012, pp. 13–16.
- [23] Devvi Sarwinda et al. “Deep learning in image classification using residual network (ResNet) variants for detection of colorectal cancer”. In: *Procedia Computer Science* 179 (2021), pp. 423–431.

X. APPENDIX A: COMPLETED GANTT CHART



XI. APPENDIX B: TEAM MEETINGS AND FACULTY MEETINGS

A. Team Meetings

- **09/19/2025:**
 - Discussion with graduate student Liz Rand about the project. Going over key-aspects, challenges, and lessons learned from last year.
- **09/22/2025:**
 - Reviewing Gantt chart and establishing goals for the fall semester. Focusing primarily on developing our spiking neural network.
- **09/26/2025:**
 - Worked on finalizing the project proposal.
 - Established a Github to push code.
- **10/04/2025:**
 - Research about how SNNs work.
 - Started ReRAM research.
 - Deciding between two datasets: Google Speech Commands Dataset v0.02 and OpenVoc Keyword Spotting Research Datasets.
 - Started the encoding process for the SNN.
- **10/06/2025:**
 - Continued working on the encoding of the SNN.
 - Worked on trimming the dataset and implementing background noise.
- **10/18/2025:**
 - Finished mixing dataset with background noise.
- **10/19/2025:**
 - Continued working on code for SNN.
 - Worked on progress report.
- **10/21/2025**
 - Finished progress report.
- **10/25/2025**
 - Revised dataset for proper usage.
- **10/29/2025**
 - Worked on presentation.
- **10/30/2025**
 - Researched tapeout companies such as Muse Semiconductor.
- **11/04/2025**
 - Continued working on presentation.
- **11/06/2025**
 - Created raster plots from the audio samples.
 - Discussed about how to train the SNN.
- **11/07/2025**
 - Worked on training the SNN.
 - Researched about ReRAM.
- **11/08/2025**
 - Continued training of the SNN.
 - Continued research about ReRAM.
- **11/09/2025**
 - Developed a block diagram of ReRAM function.
 - Continued working on SNN.
- **11/10/2025**
 - Finalized presentation and practicing.
 - Research about SNN.
- **11/11/2025**
 - Practice presentation.
- **11/21/2025**
 - Started working on making an ANN.
- **11/23/2025**
 - Continued working on the ANN.
- **11/24/2025**
 - Finished first version of the ANN.

- **11/30/2025**
 - Second version of the ANN was developed.
 - **12/03/2025**
 - Third version of the ANN was developed.
 - **12/06/2025**
 - Fourth version of the ANN was developed.
 - **12/09/2025**
 - Worked on a draft for the interim report.
 - **02/07/2026:**
 - Shifted to only focusing on the ANN2SNN conversion.
 - Implemented a ResNet10 architecture.
 - **02/14/2026:**
 - Discussed the architecture and how to make the core layout.
 - **02/20/2026:**
 - Finalized an initial three core layout.
 - **02/28/2026:**
 - Developed slides showcasing the current position of the project.
 - **03/05/2026:**
 - Changed the core layout from differential column to differential row.
 - **03/15/2026:**
 - Looked over the PDK in Cadence Virtuoso.
 - **03/24/2026:**
 - Running experiments with a fully quantized network and different bit quantization.
 - **03/28/2026:**
 - Developed SKILL scripts to place all 1T1R circuitry.
 - **04/01/2026:**
 - Finished mapping out the first core.
 - Discussed the circuitry for the multiplexers.
 - **04/04/2026:**
 - Developed an initial draft of the neuron circuitry.
 - Encountered simulation errors in Cadence Virtuoso.
 - **04/07/2026:**
 - Started developing slides for oral presentation.
 - Swapped conductance and resistor mapping as it was previously wrong.
 - Discussion of what time steps to use for the neural network.
 - **04/09/2026:**
 - Ran into a recurring simulation issue in Cadence Virtuoso.
 - Started shifting the ResNet architecture to a general CNN.
 - Tested varying initial channels, layers, and bit quantization to compare accuracy and neuron count.
 - **04/11/2026:**
 - Continued working on oral presentation slides.
 - Finalized 1T1R cells on all three cores on Cadence Virtuoso.
 - Finished initial draft of oral presentation slides.
 - **04/16/2026:**
 - Revisions on oral presentation slides.
 - Started developing sections of the poster.
 - **04/19/2026:**
 - Practiced for oral presentation.
 - Made more revisions on oral presentation slides.
 - **04/24/2026:**
 - Revised poster and discussed what to include.
 - **05/01/2026:**
 - Started drafting the final report.
 - Simulated neuron circuitry correctly. Began implementing circuitry beyond the neuron.
- B. Faculty Meetings*
- **10/01/2025:**
 - Recommended that we find a dataset that uses single word commands instead of full sentences. Transitioning away from the MS-SNSD and instead using the Google Speech Commands Dataset v0.02. Utilizing snnTorch through PyTorch to develop our SNN.

- **10/09/2025:**
 - Suggested trying out Google Speech Commands Dataset v0.02. The dataset can just be trimmed, no need for balanced sampling.
- **10/15/2025:**
 - No current graduate students available until next month to provide additional advising.
- **10/22/2025:**
 - Provided suggestions for our proposal, suggested more detail in the Gantt chart.
 - Explained that the encoding will need more time for testing, and that we should benchmark it to measure accuracy.
 - A paper was given talking about ReRAM.
- **10/29/2025:**
 - Mentioned short term FFT to spikes. For the hardware, we should start with the array.
 - Prof. Stanacevic will provide us the email of a graduate student who might be able to assist us with the neural network.
 - For things to add include in the upcoming presentation: reference to background from proposal, energy efficiency explanation, implementation of the weights, and analog and digital signals.
- **11/05/2025:**
 - Talked about target power consumption, it is currently hard to tell until we get to the cells. It will be determined once array size is finalized. Noted that it should be in microWatts.
 - The CMOS tech that will be used is the SkyWater ReRam 130 nm PDK.
 - Discussed the ReRAM paper, provided explanation about figures about a single core and its CMOS neurons.
 - Suggested adding illustrations to our presentation. This includes topics about fetching data, spiking neurons, dataset, and ReRam cell integration.
 - Mentioned adding references to the last slide.
- **11/12/2025**
 - Discussed ADC digital numbers being 4 bits. For now we would be doing max bit resolution for now and can adjust when needed (at some point the effect of the bit resolution will plateau).
 - Asked about differential input and outputs and if it is only concerning multi-bits.
- Talked about tapeout, saying we can do it for a singular core, getting multiple chips. The benefit of a SNN is that digital signals are easy to route.
- Asked about how implementation works if a certain layer is too big and requires two cores. Suggested that for now we would first find the number of neurons for an ANN. The conversion depends on if our network will be fully connected which we can specify this in our training.
- Confirmed we need a spike generation module between the ADC and next layer. The inputs of the next layer will be an input of spikes, not multi-bits.
- Only worry about the core for now, we can do the controller outside the digital logic chip.
- For now no pre- and post-processor for flexibility in converting spikes.
- Obtained email of graduate student.
- **11/20/2025:**
 - Cadence license will be obtained after Thanksgiving week.
 - For now there is no tapeout date
 - Suggested to make an ANN before going to SNN
 - Mentioned to look into making the controller in the form of a microcontroller
 - Suggested we look into how values are stored in the cells and understand how neurons are combined at the bit level.
- **11/26/2025:**
 - Developed an ANN that currently has 54% accuracy for 50 epochs. Suggested to reach 90% with ANN.
- **12/03/2025:**
 - Cadence license is not ready.
 - The ANN had an 85% test but 99% training, suggested making the gap smaller and in the 90s.
 - Discussed the details about the ANN such as the architecture, the amount of layers, and the number of neurons.
- **12/11/2025:**
 - Cadence license will be ready over break
 - Write about how the architecture will effect software and hardware for best worst case design.
 - Get the architecture done as soon as possible.

- Try less noise, clean data, or less layers to see how the model operates. Then, see how to prevent overfitting.
- Would be nice to see which noise gives the most issues.
- **02/02/2026:** - Discussed about reducing the current SNN's neurons. PDK has not been acquired yet.
- **02/09/2026:**
 - Clarified what a neuron is. Experimented with different channel numbers and layers.
 - Implemented an 8-bit quantization for the spectrogram inputs.
 - Try to create a direct SNN if possible. Chip architecture should be done by next meeting.
- **02/11/2026:**
 - Finalized the channel number and layers to be 16 and 3 respectively.
- **02/18/2026:**
 - Going with a three core layout.
 - Need to design the routing for ease.
- **02/25/2026:**
 - Figure out what exactly to draw the cell. Make slides summarizing the project progress.
- **03/02/2026:**
 - Explain how the conversion from ANN to SNN happens with SpikingJelly.
 - Start thinking about how to explain the data moving through the convolution layers.
- **03/09/2026:**
 - Clarify on the time cycles for the SNN. When there is enough data gathered, it should be sent immediately.
- **03/23/2026:**
 - Fixed technical difficulties regarding the PDK and Cadence Virtuoso.
 - Apply resistance values of 300 k Ω and 10 k Ω to the conductances for mapping.
- **04/02/2026:**
 - Discussed about implementing high and low state resistor values into Cadence Virtuoso using SKILL
- Discussed about the internal circuit of the integration amplifier. Suggested a CMOS differential amplifier to test it out
- Discussed about the multiplexers. Put this aside for now.
- **04/06/2026:**
 - Sweeping error during simulation. Suggested including the input file.
- **04/13/2026:**
 - Discussed revisions for oral presentation.
- **04/20/2026:**
 - Discussed simulation error with the neuron, even with input file.
 - Attempted 1T1R cell design in virtuoso. Noted missing DRC.
 - Discussed resistor library for layout.
 - Set resistor values using the two sheet resistances library.
 - Attempted to run a simple amplifier to determine if simulation error still persisted.
- **04/27/2026:**
 - Discussion about simulation error and the correct initializations.
 - Simulation success for neuron.
 - Begin designing the circuits beyond a neuron, along with the circuit to generate the pulses along rows.
 - Utilize as a cell just a single resistor for now and replace with memristor later on.
 - Run a parametric analysis of different widths of the time pulse to benchmark the neuron circuit.
- **05/04/2026:**
 - Replace two-input amplifier with a one-input
 - Focus on the multiplexers after the latch

XII. APPENDIX C: TEAM DISCUSSIONS AND RESUMES

A. Collaborative and Inclusive Environment

A successful environment was established by holding regular meetings to discuss progress, challenges, and design decisions. Each member was encouraged to contribute ideas regardless of their specific role or the stage of the project. Tasks were divided based on individual

strengths while still keeping everyone involved in major decisions. Regardless of one’s area of focus, everyone provided feedback on both the software and hardware aspects of the project. This approach helped improve the technical quality of the project while also strengthening teamwork and communication skills.

B. Contributions

Spencer Wu (Software & Hardware)

- Developed and curated the mixed dataset used throughout project.
- Spearheaded the development of the ANN2SNN conversion framework and neuron design, validating functionality through simulation.
- Created SKILL scripts to automate layout tasks across all cores.

Arhaam Hossain (Hardware)

- Conducted hardware research on ReRAM-based memory arrays and low-power circuit design.
- Investigated tapeout feasibility and design constraints of ReRAM cells.
- Designed a block diagram illustrating the working principles and operational states of a ReRAM memory cell.

Huabin Wu (Software)

- Performed foundational research into SNNs, investigating core architectural principles and evaluating software frameworks suitable for model development and deployment.
- Assisted in the development of spectrograms from audio samples.
- Optimized the SNN hyperparameters and architectural configurations to balance model accuracy with hardware target constraints

Ryan Lin (Software)

- Collaborate on encoding, training, and preprocessing of input data.
- Lead the development of the training process for the SNN.
- Provide ongoing support for system integration, troubleshooting, and performance.

C. Resumes

Resumes can be seen attached in the following pages.